

**Міністерство освіти і науки України  
Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»  
Факультет інформатики та обчислювальної техніки  
Кафедра обчислювальної техніки**

**Лабораторна робота №2.**

з дисципліни

«Об'єктно-орієнтоване програмування»

на тему

«Розробка графічного редактора об'єктів »

Виконала:

студентка групи ІМ-53

Кіосова Софія Олександрівна

Номер в списку - 13

Перевірів:

Рекечинський Д. О

Київ 2020

**Завдання**

1. Створити у середовищі розробки новий проект з ім'ям Lab2.
2. Скопіювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми

### **Завдання згідно варіанту**

1. У звіті повинна бути схема успадкування класів – діаграма класів.
2. Для вибору типу об'єкта в графічному редакторі Lab2 повинно бути меню "Об'єкти" з чотирма підпунктами. Меню "Об'єкти" повинно бути праворуч меню "Файл" та ліворуч меню "Довідка". Підпункти меню "Об'єкти" містять назви українською мовою геометричних форм. Геометричні форми згідно варіанту завдання
3. Масив вказівників для динамічних об'єктів типу Shape N=113
4. Гумовий" слід при вводі об'єктів суцільна лінія червоного кольору для варіантів ( $J \bmod 4 = 1$ )
5. 6.1. Прямокутник (Увід - від центру до одного з кутів для варіантів ( $J \bmod 2 = 1$ )) (Відображення - чорний контур прямокутника без заповнення для ( $J \bmod 5 = 3$  або 4)) (Кольори заповнення - світло-зелений для ( $J \bmod 6 = 1$ ))
6. Еліпс (Увід - по двом протилежним кутам охоплюючого прямокутника для варіантів ( $J \bmod 2 = 1$ )) (Відображення - чорний контур з кольоровим заповненням для ( $J \bmod 5 = 3$  або 4)) (Кольори заповнення - жовтий для ( $J \bmod 6 = 1$ ))
7. Позначка поточного типу об'єкту, що вводиться в заголовок вікна для ( $J \bmod 2 = 1$ )

( $J = 13$ ) буде:

- статический массив для Shape ( $13 \bmod 3 = 1$ ) обсягом 113 об'єктів
- "гумовий" слід ( $13 \bmod 4 = 1$ ) – суцільна лінія червоного кольору
- прямокутник: - Увід від центру до одного з кутів ( $13 \bmod 2 = 1$ )
- Відображення - чорний контур прямокутника без заповнення ( $J \bmod 5 = 3$ )
- Кольори заповнення - світло-зелений ( $J \bmod 6 = 1$ )
- еліпс: - Увід по двом протилежним кутам охоплюючого прямокутника ( $J \bmod 2 = 1$ ))
- Відображення чорний контур з кольоровим заповненням ( $J \bmod 5 = 3$  або 4)
- колір заповнення: жовтий ( $J \bmod 6 = 1$ ))
- Позначка поточного типу об'єкту, що вводиться в заголовок вікна для ( $J \bmod 2 = 1$ )

### **Структура проекту**

- [main.py](#) – головний файл, точка входу у програму
- [editor.py](#) – Логіка малювання
- [shapes.py](#) – Модель даних
- [window.py](#) – Побудова графічного інтерфейсу

## Код

### [main.py](#)

```
import tkinter as tk

from window import Window

if __name__ == "__main__":
    root = tk.Tk()

    root.geometry("800x600")

    app = Window(root)

    root.mainloop()
```

### [editor.py](#)

```
from shapes import Shape, Point, Line, Rectangle, Ellipse

class Editor():
    def __init__(self, canvas):
        self.canvas = canvas

        self.MAX_SHAPES = 113

        self.shapes = []

        self.current_shape_type = "Rectangle"

        self.start_x = 0

        self.start_y = 0

        self.rubber_line_id = None

        self.is_drawing = False
```

```
self.canvas.bind('<ButtonPress-1>', self.on_press)

self.canvas.bind('<B1-Motion>', self.on_drag)

self.canvas.bind('<ButtonRelease-1>', self.on_release)


def set_shape_type(self, shape_type):

    self.current_shape_type = shape_type

    self.is_drawing = False

    self.start_x = 0

    self.start_y = 0

    if self.rubber_line_id:

        self.canvas.delete(self.rubber_line_id)

        self.rubber_line_id = None


def on_press(self, event):

    self.is_drawing = True

    self.start_x = event.x

    self.start_y = event.y


def on_drag(self, event):

    if not self.is_drawing:

        return

    if self.rubber_line_id:

        self.canvas.delete(self.rubber_line_id)

    temp_shape = None

    if self.current_shape_type == "Point":

        temp_shape = Point(self.start_x, self.start_y, event.x,
event.y)

    elif self.current_shape_type == "Line":
```

```
        temp_shape = Line(self.start_x, self.start_y, event.x,
event.y)

        elif self.current_shape_type == "Rectangle":

            temp_shape = Rectangle(self.start_x, self.start_y, event.x,
event.y)

            elif self.current_shape_type == "Ellipse":

                temp_shape = Ellipse(self.start_x, self.start_y, event.x,
event.y)


        if temp_shape:

            self.rubber_line_id =
temp_shape.draw_rubber_band(self.canvas)

            self.rubber_line_id = Shape.draw_rubber_band(

                self.canvas, self.start_x, self.start_y, event.x, event.y

            )


def on_release(self, event):

    if not self.is_drawing:

        return

    self.is_drawing = False

    if self.rubber_line_id:

        self.canvas.delete(self.rubber_line_id)

        self.rubber_line_id = None


    shape_obj = None

    if len(self.shapes) >= self.MAX_SHAPES:

        print(f"Досягнуто ліміту об'єктів ({self.MAX_SHAPES})!")

        return
```

```

        if self.current_shape_type == "Point":
            shape_obj = Point(self.start_x, self.start_y)

        elif self.current_shape_type == "Line":
            shape_obj = Line(self.start_x, self.start_y, event.x,
event.y)

        elif self.current_shape_type == "Rectangle":
            shape_obj = Rectangle(self.start_x, self.start_y, event.x,
event.y)

        elif self.current_shape_type == "Ellipse":
            shape_obj = Ellipse(self.start_x, self.start_y, event.x,
event.y)

        if shape_obj:
            shape_obj.draw(self.canvas)

            self.shapes.append(shape_obj)

            print(f"Додано об'єкт {self.current_shape_type}. Всього у
масиві: {len(self.shapes)}/{self.MAX_SHAPES}")

```

### [shapes.py](#)

```

from abc import ABC, abstractmethod

class Shape(ABC):

    def __init__(self, x1, y1, x2=0, y2=0):

        self.x1 = x1

        self.y1 = y1

        self.x2 = x2

        self.y2 = y2

    @abstractmethod

    def draw(self, canvas):

```

```

        pass

    @abstractmethod
    def draw_rubber_band(self, canvas):
        pass

class Point(Shape):
    def draw(self, canvas):
        return canvas.create_oval(self.x1 - 2, self.y1 - 2, self.x1 +
2, self.y1 + 2, fill="black")

    def draw_rubber_band(self, canvas):
        return canvas.create_ovalself(self.x1 - 2, self.y1 - 2, self.x1
+ 2, self.y1 + 2, fill="red")

class Line(Shape):
    def draw(self, canvas):
        return canvas.create_line(self.x1, self.y1, self.x2, self.y2,
fill="black")

    def draw_rubber_band(self, canvas):
        return canvas.create_line(self.x1, self.y1, self.x2, self.y2,
fill="red", dash=(4, 2))

class Rectangle(Shape):
    def draw(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        return canvas.create_rectangle(
            self.x1 - dx, self.y1 - dy,
            self.x1 + dx, self.y1 + dy,
            outline="black", fill="#90EE90"

```

```

    )

    def draw_rubber_band(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        return canvas.create_rectangle(
            self.x1 - dx, self.y1 - dy,
            self.x1 + dx, self.y1 + dy,
            outline="red", dash=(4, 2)
        )

class Ellipse(Shape):
    def draw(self, canvas):
        return canvas.create_oval(
            self.x1, self.y1,
            self.x2, self.y2,
            outline="black", fill="#FFFB01"
        )

    def draw_rubber_band(self, canvas):
        return canvas.create_oval(
            self.x1, self.y1,
            self.x2, self.y2,
            outline="red", dash=(4, 2)
        )

```

[window.py](#)

```
import tkinter as tk
```

```
from editor import Editor

class Window():

    def __init__(self, root):

        self.root = root

        self.root.title("ООП_lab2 - [Прямокутник]")

        root.geometry("600x400")

        canvas = tk.Canvas(root, bg="lightgray")

        canvas.pack(fill=tk.BOTH, expand=True)

        self.editor = Editor(canvas)

        self.create_menu()

    def create_menu(self):

        menubar = tk.Menu(self.root)

        self.root.config(menu=menubar)

        file_menu = tk.Menu(menubar, tearoff=0)

        file_menu.add_command(label="Вихід", command=self.root.quit)

        menubar.add_cascade(label="Файл", menu=file_menu)

        objects_menu = tk.Menu(menubar, tearoff=0)

        objects_menu = tk.Menu(menubar, tearoff=0)

        objects_menu.add_command(label="Крпка", command=lambda:
self.select_shape("Point"))

        objects_menu.add_command(label="Лінія", command=lambda:
self.select_shape("Line"))

        objects_menu.add_command(label="Прямокутник", command=lambda:
self.select_shape("Rectangle"))

        objects_menu.add_command(label="Еліпс", command=lambda:
self.select_shape("Ellipse"))

        menubar.add_cascade(label="Об'єкти", menu=objects_menu)
```

```
        help_menu = tk.Menu(menubar, tearoff=0)

        help_menu.add_command(label="Про програму",
command=self.show_about)

        menubar.add_cascade(label="Довідка", menu=help_menu)


def select_shape(self, type):

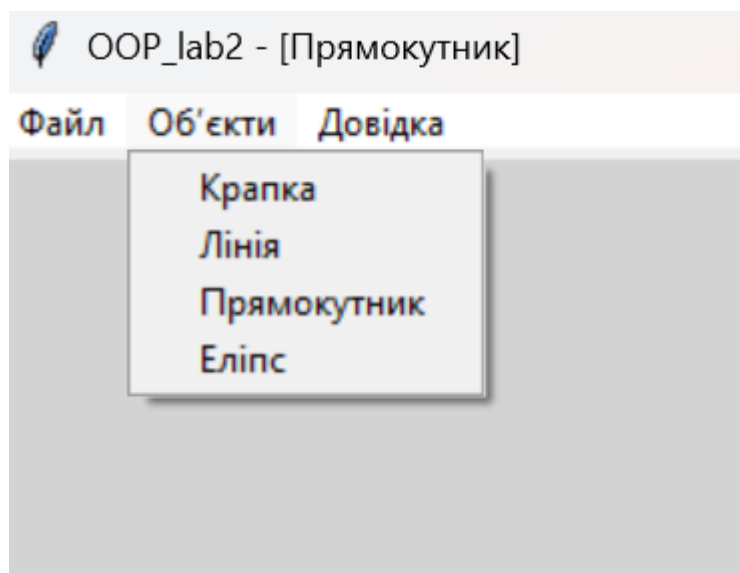
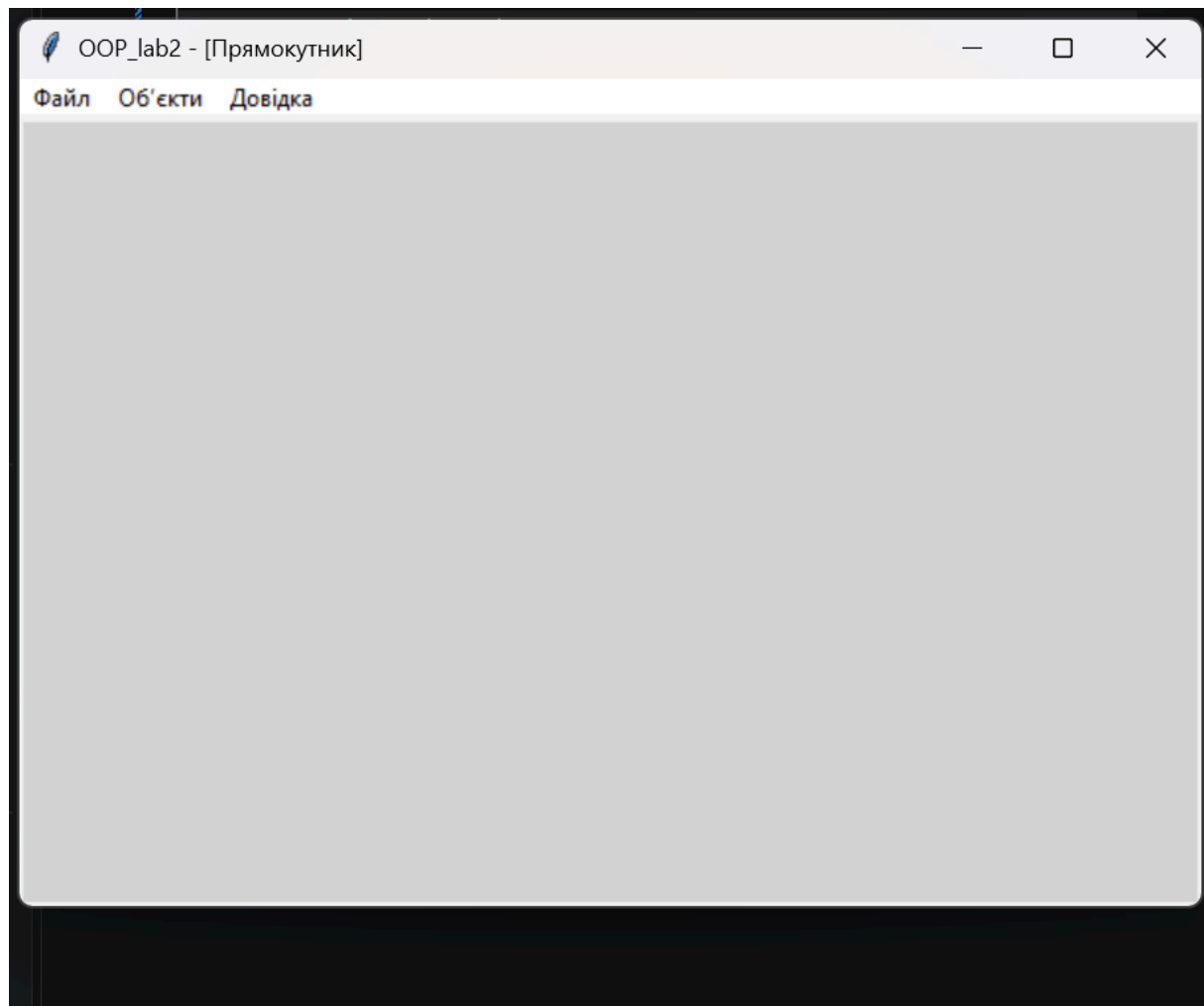
    self.editor.set_shape_type(type)

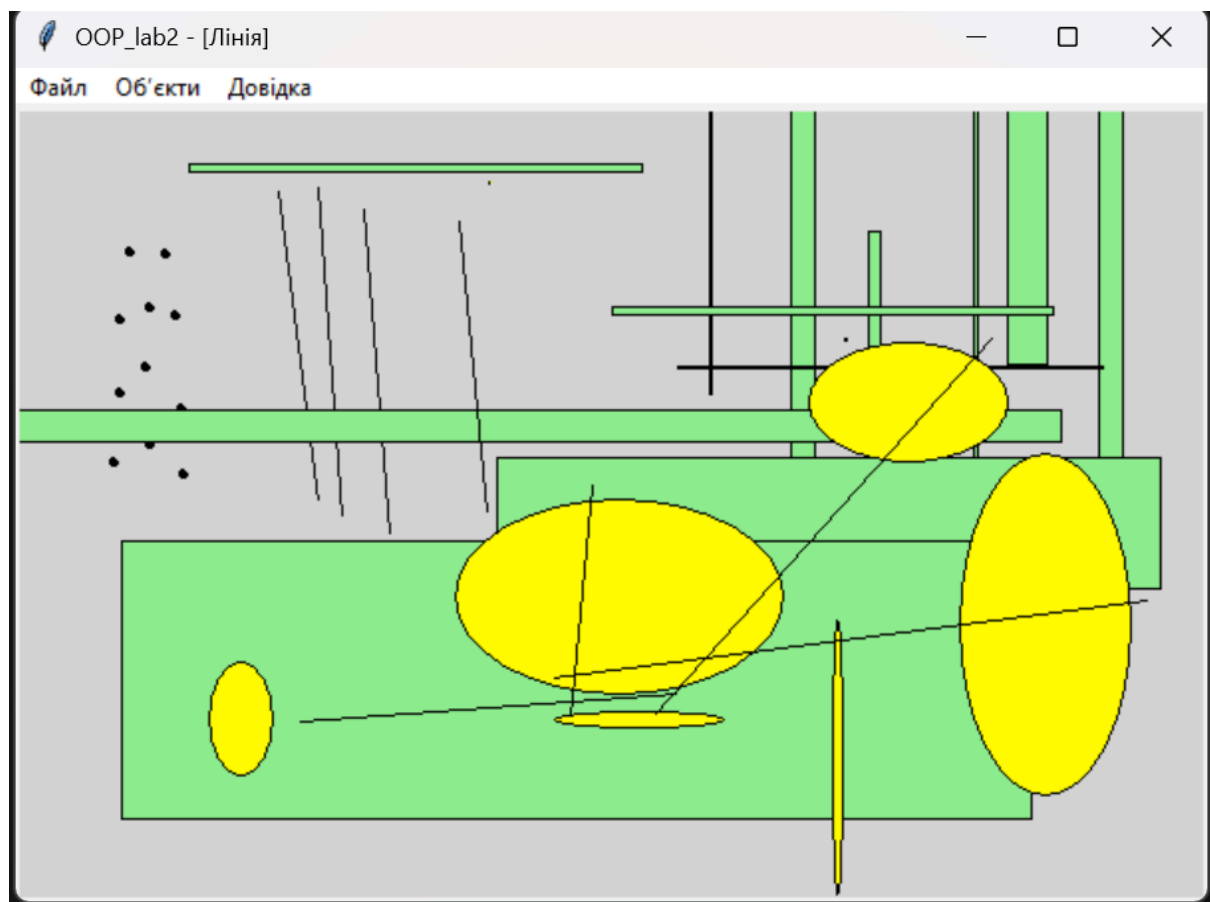
    self.root.title(f"ООП_lab2 - {type}")


def show_about(self):

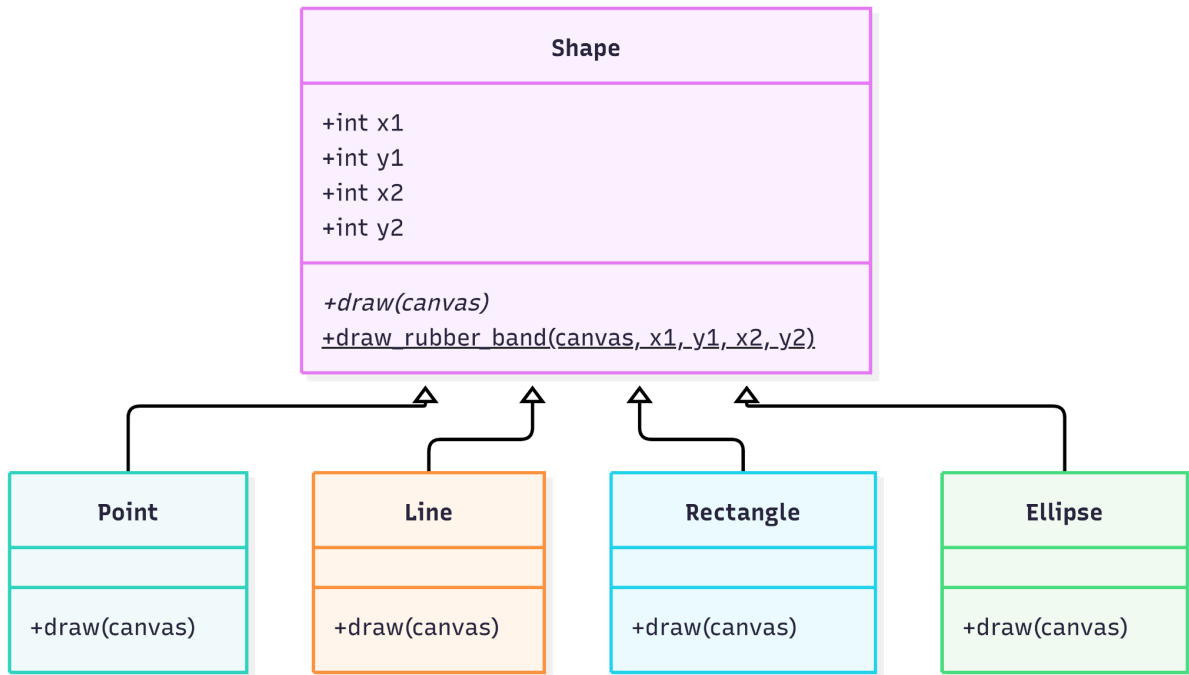
    print("Лабораторна робота №2 з ООП")
```

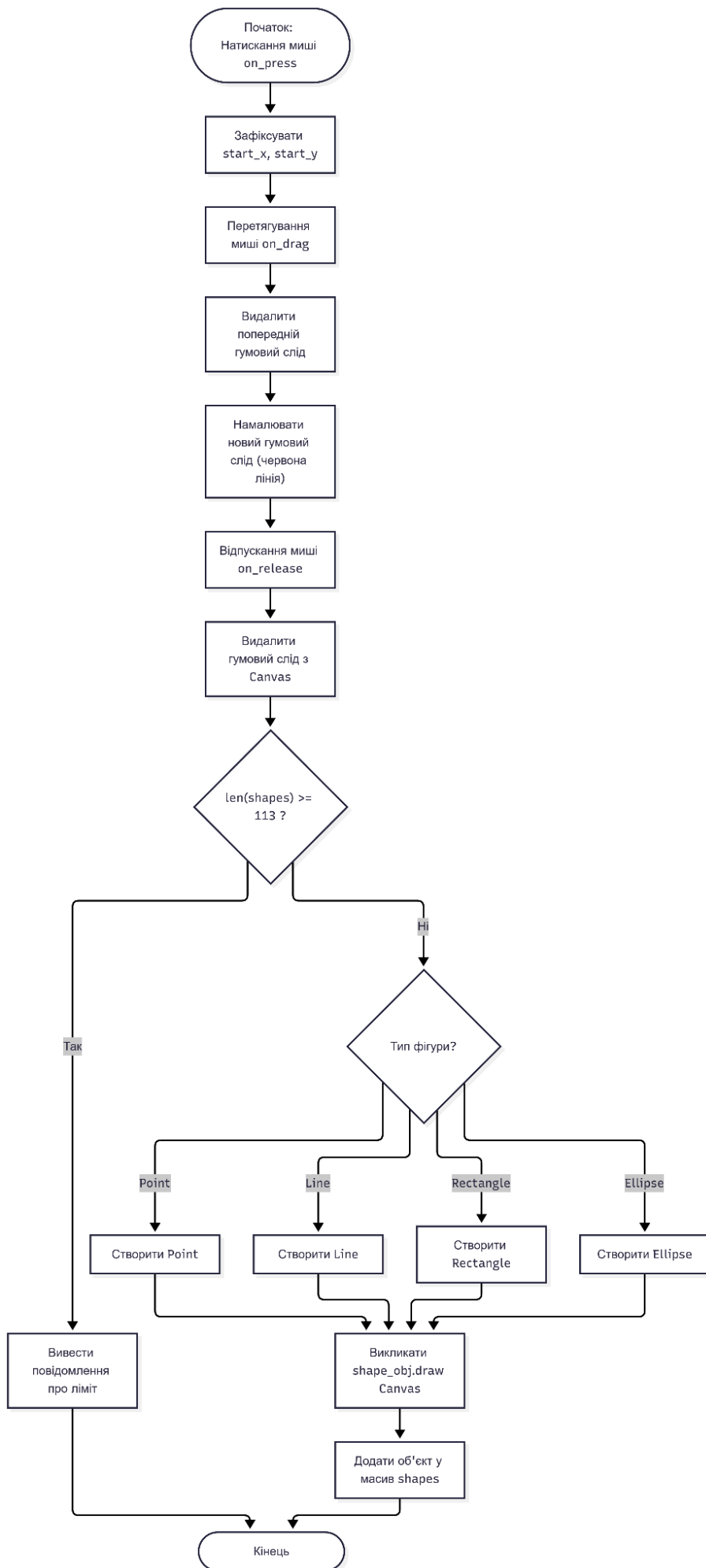
## Скріншоти





**Блок-схеми**





**Висновок:**

Навчилися використовувати абстрактні класи та успадкування. Реалізували поліморфізм, що дозволило редактору взаємодіяти з усіма об'єктами однаково, а кожна фігура сама знає, як намалювати свій власний контур та ескіз під час руху миші. Закріпили навички роботи з Tkinter та модульності